Banik_Assignment3

Arpa Banik

2024-02-26

QUESTION 1

Household Number	Income	Lot Size	Ownership or Riding Mower	Euclidean distances (Income)	Euclidean distances (Lot Size)
1	-0.42	-0.25	Owner	0.02	0.65
2	0.89	-0.92	Owner	1.29	1.32
3	-0.17	1.09	Owner	0.23	0.69
4	-0.34	0.76	Owner	0.06	0.36
5	0.35	0.25	Nonowner	0.75	0.15
6	-0.79	0.76	Nonowner	0.39	0.36
7	-0.17	-0.75	Nonowner	0.23	1.15
8	0.82	-0.58	Nonowner	1.22	0.98

Question 1 data

a. k=5

Household	Ownership
1	Owner
3	Owner
4	Owner
6	Nonowner
7	Nonowner

1a

Probability of owner: 0.6 Probability of Nonowner: 0.4 To classify the new household, I choose the class with the highest probability. Since 0.6>0.4, I classified the new household as an owner based on the majority class of its 5 nearest neighbors.

b. k=3

Household	Ownership
4	Owner
5	Nonowner
6	Nonowner

1b

Probability of owner: 1/3 Probability of Nonowner: 2/3 To classify the new household, I choose the class with the highest probability. Since 2/3>1/3, I classified the new household as a non-owner based on the majority class of its 3 nearest neighbors using lot size as the predictor.

c.

Household	Ownership
1	Owner
3	Owner
4	Owner
5	Nonowner
6	Nonowner
7	Nonowner
8	Nonowner

1c

Out of the 7 nearest neighbors, 3 are owners and 4 are non-owners. Therefore, the probability of the new household being in the owner class is 3/7 and the probability of being in the non-owner class is 4/7.

To classify the new household, I choose the class with the highest probability. Since 4/7>3/7, I classified the new household as a non-owner based on the majority class of its 7 nearest neighbors.

QUESTION 2

```
#setwd("/Users/arpabanik/Library/CloudStorage/OneDrive-BentleyUniversity/R-Project 347/MA 347/Assignment 3")

# Load required libraries
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(FNN)

#Load data
data <- read.csv("~/Library/CloudStorage/OneDrive-BentleyUniversity/R-Projects/MA 347/Assignment 3/UniversalBank
(2).csv")
data <- data[ -c(1,5,8,11:14) ]
```

a. Partition the data into training (80%) and validation (20%) sets

```
train.index <- sample(c(1:dim(data)[1]), dim(data)[1]*0.8)
train_data <- data[train.index, ]
valid_data <- data[-train.index, ]

train.norm.df <- train_data
valid.norm.df <- valid_data

norm.values <- preProcess (train_data[, 1:6], method=c("center","scale"))
train.norm.df[, 1:6] <- predict(norm.values, train_data[, 1:6])
valid.norm.df[, 1:6] <- predict(norm.values, valid_data[, 1:6])

#k=1
nn <- knn(train = train.norm.df[, 1:6], test = valid.norm.df[,1:6],
            cl = train.norm.df[, 7], k = 1)
#The predicted class labels are stored in vector x
x<-as.character(nn)
```

a. continued…

```
# Set seed values for reproducibility
seed.val <- c(111, 121, 131, 141, 151)

# Initialize variables to store results
confusion_matrices <- list()
misclassification_rates <- numeric(5)

for (i in 1:5) {
  set.seed(seed.val[i])

  # Partition the data into training (80%) and validation (20%) sets
  train.index <- sample(c(1:dim(data)[1]), dim(data)[1] * 0.8)
  train_data <- data[train.index, ]
  valid_data <- data[-train.index, ]

  train.norm.df <- train_data
  valid.norm.df <- valid_data

  norm.values <- preProcess(train_data[, 1:6], method = c("center", "scale"))
  train.norm.df[, 1:6] <- predict(norm.values, train_data[, 1:6])
  valid.norm.df[, 1:6] <- predict(norm.values, valid_data[, 1:6])

  # k-NN classification with k = 1
  nn <- knn(train = train.norm.df[, 1:6], test = valid.norm.df[, 1:6],
            cl = train.norm.df[, 7], k = 1)

  # Confusion matrix for each run
  confusion_matrix <- table(nn, valid.norm.df[, 7])
  confusion_matrices[[i]] <- confusion_matrix

  # Misclassification rate for each run
  misclassification_rate <- 1 - sum(diag(confusion_matrix)) / sum(confusion_matrix)
  misclassification_rates[i] <- misclassification_rate

  cat("Confusion Matrix - Run", i, ":\n")
  print(confusion_matrix)
  cat("Misclassification Rate - Run", i, ":", misclassification_rate, "\n\n")
}
```

```
## Confusion Matrix - Run 1 :
##
## nn    0   1
##   0 884  40
##   1  17  59
## Misclassification Rate - Run 1 : 0.057
##
## Confusion Matrix - Run 2 :
##
## nn    0   1
##   0 881  42
##   1  23  54
## Misclassification Rate - Run 2 : 0.065
##
## Confusion Matrix - Run 3 :
##
## nn    0   1
##   0 892  30
##   1  16  62
## Misclassification Rate - Run 3 : 0.046
##
## Confusion Matrix - Run 4 :
##
## nn    0   1
##   0 899  33
##   1  17  51
## Misclassification Rate - Run 4 : 0.05
##
## Confusion Matrix - Run 5 :
##
## nn    0   1
##   0 884  44
##   1  14  58
## Misclassification Rate - Run 5 : 0.058
```

```
# Average misclassification rate from 5 runs
average_misclassification_rate <- mean(misclassification_rates)
cat("Average Misclassification Rate:", average_misclassification_rate, "\n")
```

```
## Average Misclassification Rate: 0.0552
```

b. Repeat for different values of k

```
k_values <- c(3, 5, 7, 9)
results_table <- matrix(0, nrow = length(k_values), ncol = 7)
colnames(results_table) <- c("k", "1st run", "2nd run", "3rd run", "4th run", "5th run", "Average")

for (j in 1:length(k_values)) {
  k <- k_values[j]
  misclassification_rates <- numeric(5)

  for (i in 1:5) {
    set.seed(seed.val[i])

    # Partition the data into training (80%) and validation (20%) sets
    train.index <- sample(c(1:dim(data)[1]), dim(data)[1] * 0.8)
    train_data <- data[train.index, ]
    valid_data <- data[-train.index, ]

    train.norm.df <- train_data
    valid.norm.df <- valid_data

    norm.values <- preProcess(train_data[, 1:6], method = c("center", "scale"))
    train.norm.df[, 1:6] <- predict(norm.values, train_data[, 1:6])
    valid.norm.df[, 1:6] <- predict(norm.values, valid_data[, 1:6])

    # k-NN classification with current k value
    nn <- knn(train = train.norm.df[, 1:6], test = valid.norm.df[, 1:6],
              cl = train.norm.df[, 7], k = k)

    # Confusion matrix for each run
    confusion_matrix <- table(nn, valid.norm.df[, 7])
    misclassification_rate <- 1 - sum(diag(confusion_matrix)) / sum(confusion_matrix)
    misclassification_rates[i] <- misclassification_rate
  }

  # Average misclassification rate for each k
  average_misclassification_rate <- mean(misclassification_rates)

  # Store results in the table
  results_table[j, ] <- c(k, misclassification_rates, average_misclassification_rate)
}

# Display results table
print(results_table)
```

```
##      k 1st run 2nd run 3rd run 4th run 5th run Average
## [1,] 3   0.060   0.061   0.049   0.052   0.058  0.0560
## [2,] 5   0.059   0.060   0.044   0.052   0.059  0.0548
## [3,] 7   0.058   0.062   0.055   0.050   0.065  0.0580
## [4,] 9   0.060   0.063   0.055   0.052   0.071  0.0602
```

c. Recommend the choice of k based on the results

```
recommended_k <- results_table[which.min(results_table[, "Average"]), "k"]
cat("\nRecommended value of k:", recommended_k, "\n")
```

```
##
## Recommended value of k: 5
```

d. Classify the entire dataset with the recommended k value

```
# Define predictors and response based on the dataset
predictors <- c("Age", "Experience", "Income", "Family", "CCAvg", "Mortgage")
response <- "Personal.Loan"
# optimal k value
k <- 5
# Normalize the predictors
preProcValues <- preProcess(data[, predictors], method = c("center", "scale"))
data_norm <- predict(preProcValues, data[, predictors])
# New customer data
new_customer <- data.frame(Age = 40, Experience = 10, Income = 84, Family = 2, CCAvg = 2, Mortgage = 0)
new_customer_norm <- predict(preProcValues, new_customer)
# Classify the new customer
knn_pred <- knn(train = data_norm, test = new_customer_norm, cl = data[, response], k = k)
knn_pred
```

```
## [1] 0
## attr(,"nn.index")
##      [,1] [,2] [,3] [,4] [,5]
## [1,]  701 1977 2307 4927 1343
## attr(,"nn.dist")
##           [,1]     [,2]      [,3]      [,4]      [,5]
## [1,] 0.2986486 0.310951 0.3897545 0.4084889 0.4207974
## Levels: 0
```

e. Calculate probabilities for the new customer

```
# Predict the class for the new customer
knn_pred <- knn(train = data_norm, test = new_customer_norm, cl = data[, response], k = k)

predicted_class <- rep(as.numeric(knn_pred), length(valid.norm.df[, 7]))

# Create a confusion matrix
conf_matrix <- table(predicted_class, valid.norm.df[, 7])

# Calculate class probabilities
class_probabilities <- conf_matrix / rowSums(conf_matrix)

# Display class probabilities
print(class_probabilities)
```

```
##
## predicted_class     0     1
##               1 0.898 0.102
```

Given these probabilities, the model is indicating a higher likelihood (0.898 probability) of the new customer rejecting the loan offer (class 0) rather than accepting it (class 1). The probabilities sum to 1, and the class with the higher probability is the one predicted by the model. Since the probability for rejection is substantially higher, the model suggests that it is more likely that the new customer will decline the loan offer if the bank makes an offer similar to the last campaign.